Dev 2 구현 가이드 — 비전공자도 이해하는 버전

작성일: 2026-05-22 **대상**: 코딩을 처음 배우거나, 개념이 낯선 분

> 이 문서는 기술 용어를 최대한 피하고 **일상 비유**로 전체 흐름을 설명합니다.

우리가 만드는 게 뭐야?

> **한 줄 요약**: 개발자가 코드를 올리면, AI가 자동으로 "이 코드 위험해요, 이 부분 고쳐요"를 알려주는 시스템

비유로 이해하기

병원 검진 시스템이라고 생각하세요.

```
환자(코드)가 병원에 옴
    → 혈액검사(Pylint), 엑스레이(Bandit), MRI(Semgrep) 등 여러 검사
    → 의사(AI/LLM)가 검사 결과 보고 최종 진단
    → 진단서(결과) 작성해서 전달
    → 기록 보관(DB)
```

전체 흐름 한눈에 보기

```
① 개발자가 GitHub에 코드 업로드
        ↓
② n8n이 감지 → "새 코드 왔어요!" 신호 전달
        ↓
③ 파일 종류(언어) 자동 감지 → 맞는 검사 도구 선택 → 동시에 검사
        ↓
④ 과거에 비슷한 문제 있었는지 검색 (Milvus)
        ↓
⑤ AI가 검사 결과 보고 심각도 판단 + 개선 방법 작성
        ↓
⑥ 위험(Critical)이면 즉시 알림
        ↓
⑦ 결과를 데이터베이스에 저장
        ↓
⑧ "이런 문제 발견됐어요" 결과 전달
```

Task 1: 주방 세팅 (Docker + 환경 세팅)

비유

식당을 열기 전에 주방 기기를 설치하는 것입니다.

```
냉장고 설치          → PostgreSQL (데이터 저장소)
재료 창고 설치       → Milvus (과거 이슈 검색 DB)
자동 주문 시스템     → n8n (GitHub 신호 받는 도구)
일정 관리 시스템     → Airflow (매일 자동 실행 도구)
주방 자체            → dev2 앱 서버
```

Docker가 뭐야?

택배 박스에 비유합니다.

```
일반 방식:
"내 컴퓨터에선 되는데 서버에선 안 돼요" 😭
→ 각자 컴퓨터 환경이 달라서 생기는 문제

Docker 방식:
앱 + 필요한 모든 것을 박스에 담아서 어디서든 똑같이 실행
→ "박스째로 실행하니까 어디서든 동일하게 동작"
```

docker-compose가 뭐야?

명령어 하나로 5개 서비스를 한꺼번에 실행하는 리모컨입니다.

```
docker-compose up   # 이 한 줄로 5개 서비스 동시 실행
```

이 Task가 끝나면?

컴퓨터에서 `docker-compose up` 명령어 하나로 모든 서비스가 켜집니다.

Task 2: 서류 양식 만들기 (Pydantic 스키마 + DB 테이블)

비유

Pydantic = 표준 업무 양식

```
"스캔 요청은 이 양식대로만 받겠습니다"

[스캔 요청서]
- 작업번호(task_id): 필수
- PR번호(pr_id): 필수
- 코드내용(pr_diff): 필수
- 개발자정보(developer): 필수
  └── 개발자ID, 실력수준, 기술스택
```

양식에 맞지 않는 요청이 오면? → 자동으로 거절합니다.

DB 테이블 = 창고 선반

```
선반 1: scan_executions (스캔 실행 기록)
    "2026-05-22 오후 3시에 PR #42 스캔함"

선반 2: scan_issues (발견된 문제 상세)
    "auth.py 42번줄에서 SQL 인젝션 위험 발견"

선반 3: scan_summaries (요약 숫자)
    "위험 2개, 경고 5개, 주의 3개"
```

TDD가 뭐야? (테스트 주도 개발)

레시피 검증에 비유합니다.

```
일반 방식:
요리 만들기 → 먹어보기 → 맛없으면 수정

TDD 방식:
"이 맛이 나야 합니다" 기준 먼저 정하기
→ 기준 충족하는 레시피 만들기
→ 자동으로 검증
```

코드로 보면:

```python
# ① 먼저 "이렇게 동작해야 한다" 테스트 작성
def test_task_payload():
    p = TaskPayload(task_id="001", pr_id="42", ...)
    assert p.developer.skill_level == "junior"  # 이래야 한다!

# ② 위 테스트를 통과하는 코드 구현
# ③ 테스트가 통과하면 완성
```

Task 3: 언어별 맞춤 검사관 팀 + 자동 배정 시스템

비유

어떤 언어로 된 코드든 해당 언어 전문 검사관에게 자동으로 배정됩니다.

```
Python 파일 (.py)   → Python 검사관 팀
JavaScript 파일(.js) → JavaScript 검사관 팀
C# 파일 (.cs)       → C# 검사관 팀
```

```
HTML 파일 (.html)   → HTML 검사관 팀
+ 어떤 언어든 → 공통 보안 검사관(Semgrep)이 항상 함께 검사
```

언어별 검사관 팀

Python 검사관 팀

검사관	전문 분야	하는 일
Pylint	코드 품질	문법 오류, 안 쓰는 변수, 네이밍 규칙 위반
radon	코드 복잡도	"이 함수 너무 복잡해서 버그 나기 쉬워요"
Bandit	Python 보안	SQL 인젝션, 하드코딩된 비밀번호 탐지
pip-audit	패키지 취약점	"쓰고 있는 라이브러리에 해킹 가능한 구멍 있어요"

JavaScript / TypeScript 검사관 팀

검사관	전문 분야	하는 일
ESLint	JS/TS 코드 품질	미사용 변수, 타입 오류, 규칙 위반
npm audit	패키지 취약점	npm 라이브러리 보안 구멍 탐지

C# 검사관 팀

검사관	전문 분야	하는 일
Roslyn Analyzer	.NET 코드 분석	C# 코드 품질 + 보안 패턴 검사

HTML / CSS 검사관 팀

검사관	전문 분야	하는 일
HTMLHint	HTML 유효성	태그 오류, XSS 취약점 탐지
Stylelint	CSS 품질	스타일 규칙 위반, 중복 선언

공통 보안 검사관 (모든 언어에 적용)

검사관	전문 분야	하는 일
Semgrep	보안 패턴	해킹 패턴 목록(OWASP Top 10)과 비교 — 언어 상관없이 전부 검사

자동 배정 시스템 (router.py)

파일 확장자를 보고 어떤 검사관 팀을 부를지 자동 결정합니다.

```
코드 파일 도착
    ↓
router.py가 확장자 확인
```

```
    ├── .py   → Python 팀 (pylint + radon + bandit + pip-audit)
    ├── .js   → JS 팀 (eslint + npm audit)
    ├── .ts   → TS 팀 (eslint + npm audit)
    ├── .cs   → C# 팀 (roslyn)
    ├── .html → HTML 팀 (htmlhint)
    └── .css  → CSS 팀 (stylelint)
         ↓
    + 항상: Semgrep (공통 보안)
```

subprocess가 뭐야?

Python이 터미널 명령어를 대신 실행해주는 것입니다.

```
터미널에서 직접:
$ pylint src/auth.py --output-format=json

Python 코드로:
result = subprocess.run(["pylint", "src/auth.py", "--output-format=json"])
# 결과를 변수에 저장해서 우리가 처리할 수 있음
```

asyncio.gather가 뭐야?

동시 주문에 비유합니다.

```
순서대로 검사 (느림):
Pylint 검사(30초) → 완료 → Bandit 검사(30초) → 완료
총 60초 걸림

asyncio.gather (빠름):
Pylint + Bandit 동시에 검사 시작
총 30초 걸림 (가장 오래 걸리는 것 기준)
```

Task 4-a: 검사관 지휘자 만들기 (run_tools 노드)

비유

Task 3에서 각 검사관을 **고용**했다면, Task 4-a는 **지휘자**를 만드는 것입니다.

```
코드가 도착하면 지휘자(run_tools 노드)가:
    1. 파일 확장자를 본다 (.py? .js? .html?)
    2. router.py에게 "어느 팀 불러야 해?" 물어본다
    3. 해당 언어 팀을 동시에 소집한다 (asyncio.gather)
    4. 결과를 바구니에 담는다
```

왜 별도 노드가 필요해?

```
Task 3 = 검사관 채용 (도구 함수 만들기)
Task 4-a = 지휘자 역할 (LangGraph 노드로 묶기)

예시:
run_tools 노드가 없으면:
    → 도구들이 있어도 누가 언제 어떻게 부를지 모름

run_tools 노드 있으면:
    → LangGraph 흐름에 연결됨
    → PR diff에서 파일 목록 추출
    → router.py로 언어 감지
    → 병렬 실행
    → 결과를 State 바구니에 저장
```

실제 흐름

```
PR diff 텍스트:
  "+++ b/src/auth.py"        → Python 파일 감지
  "+++ b/frontend/app.js"    → JavaScript 파일 감지
  "+++ b/views/index.html"   → HTML 파일 감지
          ↓
router.py: {"python": ["auth.py"], "javascript": ["app.js"], "html":
["index.html"]}
          ↓
asyncio.gather로 3개 언어팀 동시 소집
          ↓
static_raw: [Pylint 결과, ESLint 결과, HTMLHint 결과]
security_raw: [Bandit 결과, Semgrep(js) 결과, Semgrep(html) 결과]
```

TDD 포인트

```
# Mock(가짜 도구)을 써서 실제 CLI 없이도 테스트 가능
with patch("dev2.nodes.run_tools._run_for_language",
           return_value={"static": [이슈목록], "security": [이슈목록]}):
    result = await run_tools_node(state)
    assert len(result["static_raw"]) > 0
```

Task 4-b: AI에게 지시서 잘 쓰기 (프롬프트 엔지니어링)

비유

AI에게 막연하게 시키는 것과 구체적으로 시키는 것의 차이입니다.

```
❌ 나쁜 지시:
"이 코드 분석해줘"
→ AI가 매번 다른 기준으로 판단

✅ 좋은 지시:
"예시 3개 줄게. 이것처럼 판단해.
 그리고 1번부터 4번 순서대로 생각해서 답해.
 과거에 비슷한 사례도 참고해."
→ 일관되고 정확한 판단
```

3가지 기법

① Few-shot (예시 포함)

```
예시1: SQL 파라미터 미처리 → Critical (즉시 수정 필요)
예시2: 함수 복잡도 15 → Major (리팩토링 권고)
예시3: 변수명 규칙 위반 → Minor (다음에 수정)

→ 이 예시들처럼 판단해줘
```

② Chain-of-Thought (순서대로 생각하게)

```
"판단하기 전에 이 순서로 생각해:
 1단계: 이슈 패턴이 뭐야?
 2단계: 과거 사례와 비교하면?
 3단계: 최종 심각도는?
 4단계: 개발자 수준에 맞는 설명은?"
```

③ RAG (과거 사례 주입)

```
AI한테: "이 이슈, 과거 PR #98에서
         94% 유사한 케이스가 있었어.
         그때 이렇게 처리했거든. 참고해서 판단해줘."
→ AI가 맥락 있는 판단 가능
```

Task 4-c: AI 판사 만들기 (llm_judge 노드)

비유

검사관(CLI 도구)들이 "이상한 것 같아요"라고 보고하면, **시니어 엔지니어(AI)** 가 최종 판결을 내리는 과정입니다.

비용 아끼기: 모델 라우팅

AI API는 호출할 때마다 돈이 듭니다.

```
Critical (보안 위험) → 비싼 모델(gpt-4o) 사용
    → 정확도 최우선, 보안이니까 실수하면 안 됨

Major (코드 품질) → 저렴한 모델(gpt-4o-mini) 사용
    → 균형 잡힌 선택

Minor (스타일 위반) → AI 호출 안 함!
    → 굳이 AI 안 써도 됨, 돈 아끼기
```

코드 흐름

```
이슈 목록에서 하나씩 꺼냄
    │
    ├── Minor야? → 그냥 통과 (AI 안 부름)
    │
    └── Critical/Major야?
            │
            ├── 과거 유사 사례 + 이슈 내용을 AI에게 전달
            │
            ├── AI가 최종 심각도 + 개선 방법 작성
            │
            └── AI가 실패하면? → CLI 결과 그대로 사용 (안전장치)
```

Task 4-d: 과거 사례 검색 (vector_search 노드 — RAG)

비유

"이 증상, 예전 환자에서 본 적 있는데?"를 자동으로 찾아주는 것입니다.

임베딩이 뭐야?

텍스트를 숫자로 변환하는 것입니다.

```
"SQL 인젝션 위험" → [0.12, -0.34, 0.89, 0.02, ...]  (1536개 숫자)
"SQL injection"   → [0.11, -0.33, 0.91, 0.01, ...]  (비슷한 숫자!)
"오늘 날씨"       → [0.92,  0.15, -0.44, 0.77, ...]  (전혀 다른 숫자)
```

의미가 비슷하면 숫자도 비슷합니다 → 이걸 이용해서 "비슷한 이슈" 검색

왜 AI 판단(4-c) 전에 실행해야 해?

```
❌ 순서 바꾸면:
AI가 판단할 때 → "과거 사례? 몰라요"
→ 맥락 없이 판단

✅ 올바른 순서:
1. 먼저 유사 사례 검색 (4-d)
2. AI에게 유사 사례 알려주고 판단 (4-c)
→ "PR #98에서 비슷한 케이스가 있었어" 알고 판단
```

Task 4-e: AI 채점 능력 측정 (LLM Evaluation Harness)

비유

AI 선생님이 시험을 얼마나 잘 채점하는지 검증하는 것입니다.

```
정답지(골든 데이터셋) 만들기:
    "SQL 파라미터 미처리" → 정답: Critical
    "변수명 위반"         → 정답: Minor
    "CVE 점수 9.1"       → 정답: Critical
    (10개 준비)

AI한테 채점 시키기
    → AI 답: Critical ✅, Minor ✅, Major ❌ ...

정확도 계산:
    7/10 = 70% → 프롬프트 개선 → 9/10 = 90%
```

왜 필요해?

```
AI가 항상 맞는다는 보장이 없음
→ 측정해야 개선할 수 있음
→ 포트폴리오: "프롬프트 개선으로 정확도 70% → 90% 달성" 수치 제시 가능
```

Task 4-f: 결과 저장하기 (store 노드)

비유

검진 결과를 두 군데에 동시에 제출하는 것입니다.

```
PostgreSQL (병원 진료 기록부)
    → 지금 바로 조회 가능
    → "이 환자 오늘 검진 결과 보여줘"
```

```
BigQuery (의료 연구 데이터베이스)
    → 나중에 분석용
    → "지난 1년 동안 SQL 인젝션이 몇 번 발견됐어?"
```

asyncio.gather로 동시 저장

```python
# 순서대로 저장: 1초 + 1초 = 2초
await save_postgresql(result)
await save_bigquery(result)

# 동시에 저장: max(1초, 1초) = 1초
await asyncio.gather(
    save_postgresql(result),
    save_bigquery(result),
)
```

Task 5: 부품 조립하기 (LangGraph StateGraph)

비유

앞에서 만든 5개 부품을 설계도대로 조립하는 것입니다.

State = 작업 바구니

```
모든 노드가 공유하는 바구니:
{
    payload:      "PR #42, 개발자 dev-001"            ← Task가 시작할 때 채워짐
    static_raw:   [언어별 코드품질 도구 결과들]          ← run_tools가 채움
                  (Pylint, ESLint, Roslyn, HTMLHint 등)
    security_raw: [언어별 보안 도구 결과들]             ← run_tools가 채움
                  (Bandit, Semgrep, npm audit 등)
    enriched:     [유사 이슈 + AI 판단 결과]            ← vector_search,
llm_judge가 채움
    final_output: [최종 결과]                         ← store가 채움
}
```

조립 흐름

```
run_tools (CLI 검사)
    ↓
vector_search (유사 이슈 검색)
    ↓
llm_judge (AI 최종 판단)
    ↓
Critical 있어? ── Yes ──→ escalate (즉시 알림 발송)
```

```
    |                              ↓
    No                          store (저장)
    ↓
store (저장)
    ↓
끝 (END)
```

조건부 엣지란?

```
일반 도로: 항상 직진 (A → B → C)

조건부 도로: 신호등처럼 상황에 따라 다른 방향
    Critical 있으면 → 경보 울리고 → 저장
    Critical 없으면 → 바로 저장
```

Task 6: 창구 만들기 (FastAPI 엔드포인트)

비유

외부에서 우리 시스템에 접근하는 창구(접수처)를 만드는 것입니다.

파일 구조 (창구 3개 파일)

```
api/
├── scan.py        → POST /scan             (스캔 접수)
├── insights.py    → GET  /insights/*       (통계 조회 3종)
└── issues.py      → PATCH /issues/*/dismiss (오탐 표시)
```

> dismiss를 왜 별도 파일로 분리했을까? insights 라우터는 `/insights` 로 시작. dismiss URL은 `/issues/...` 로 시작. 같은 파일에 넣으면 URL이 `/insights/issues/...` 가 되어버림 → 별도 분리

창구 5개

```
POST /scan
    → "이 PR 검사해줘" 접수
    → "접수됐어요"(202) 즉시 답변
    → 뒤에서 조용히 검사 진행

GET /insights/developer/{id}
    → "dev-001 개발자의 문제 패턴 보여줘"
    → "SQL 인젝션 3번, 복잡도 초과 5번..."

GET /insights/team
    → "이번달 팀 전체 Top5 문제"
```

```
GET /insights/trends
    → "주차별 위험 이슈 추이"

PATCH /issues/{id}/dismiss
    → "이 이슈는 오탐이야, 무시해"
    → scan_issues.is_false_positive = true 로 업데이트
```

202 상태코드란?

```
200 OK       = "완료됐어요"
202 Accepted = "접수됐어요, 처리 중이에요"

왜 202?
검사가 30초 걸리는데 "끝날 때까지 기다려"라고 하면
연결이 끊길 수 있음.
"접수했어요, 나중에 결과 조회해요"가 맞는 방식.
```

Task 7: 자동화 스케줄러 + 연결 자동화 (Airflow + n8n)

Airflow — 자동 알람시계

```
매일 새벽 2시 알람 (daily_etl):
    "PostgreSQL에서 어제 데이터 꺼내서 BigQuery로 옮겨"

매주 월요일 오전 9시 알람 (weekly_report):
    "1. 주간 이슈 요약 만들어"
    "2. 오탐률 높은 규칙 있으면 Slack에 알려줘"
    "3. Bandit/Semgrep 최신 버전 나왔으면 알려줘"
```

Airflow DAG이 뭐야?

```
DAG = 할 일 목록 + 순서

dev2_weekly_report DAG:
    할 일 1: 주간 인사이트 집계  ┐
    할 일 2: 오탐률 집계         ↵ (1 끝나면 2 시작)
    할 일 3: 룰셋 버전 체크      ← (2 끝나면 3 시작)
```

n8n — 자동 배달부

```
GitHub에서 PR이 열림
    ↓
n8n이 감지 ("새 PR 왔어요!")
```

```
    ↓
자동으로 POST /scan 호출
    (task_id, pr_id, pr_diff, developer 정보 담아서)
```

n8n 없으면 이 연결을 직접 코딩해야 함 (수백 줄). n8n 있으면 UI에서 블록 3개 연결하면 끝.

Task 8: 전체 확인 (테스트 + E2E)

3단계 확인

1단계: 부품별 확인 (단위 테스트)

```
"llm_judge가 Minor는 AI 안 부르나?" ✅
"vector_search가 유사 이슈 잘 찾나?" ✅
각 노드를 개별로 확인
```

2단계: 조립 확인 (통합 테스트)

```
"5개 노드가 연결돼서 잘 동작하나?"
전체 흐름을 한 번에 확인
```

3단계: 실제 확인 (E2E 테스트)

```
실제로 PR 스캔 요청 → 결과 확인
curl -X POST http://localhost:8002/scan ...
curl http://localhost:8002/insights/developer/dev-001
```

전체 Task 요약표

Task	비유	핵심 기술
1. Docker 세팅	주방 기기 설치	Docker Compose
2. 스키마	서류 양식 + 창고 선반	Pydantic, PostgreSQL
3. CLI 도구	언어별 맞춤 검사관 팀 + 자동 배정	subprocess, asyncio, router.py
4a. run_tools 노드	**검사관 지휘자**	**LangGraph 노드, asyncio.gather**
4b. 프롬프트	AI 지시서 잘 쓰기	Few-shot, CoT, RAG
4c. llm_judge	AI 판사	LangChain, 모델 라우팅
4d. vector_search	과거 사례 검색	Milvus, 임베딩

Task	비유	핵심 기술
4e. Evaluation	AI 채점 능력 측정	LangSmith
4f. store	결과 두 곳에 저장	PostgreSQL, BigQuery
5. StateGraph	5개 부품 조립 (조건부 분기 포함)	LangGraph
6. FastAPI	창구 3파일 (scan/insights/issues)	FastAPI
7. Airflow + n8n	알람 + 자동 배달부	Airflow, n8n
8. 테스트	전체 확인	pytest

이해도 확인 퀴즈

1. **Q**: Docker를 쓰는 이유는? **A**: (내 컴퓨터에서만 되는 문제 방지. 박스째로 실행해서 어디서든 동일하게 동작)
2. **Q**: asyncio.gather가 없으면 어떻게 되나? **A**: (Pylint 30초 + Bandit 30초 = 60초 걸림. gather 쓰면 30초로 단축)
3. **Q**: vector_search가 llm_judge 앞에 있어야 하는 이유는? **A**: (AI가 판단할 때 "과거 유사 사례"를 알고 판단해야 더 정확하기 때문. RAG 패턴)
4. **Q**: POST /scan이 202를 반환하는 이유는? **A**: (검사가 30초 걸리는데 기다리면 연결 끊길 수 있음. "접수됐어요"로 즉시 응답)
5. **Q**: Minor 이슈는 왜 AI(LLM)를 호출하지 않나? **A**: (AI API 호출은 돈이 듦. Minor는 굳이 AI 안 써도 됨 → 비용 절감)
6. **Q**: Python 코드와 JavaScript 코드가 같이 들어오면 어떻게 되나? **A**: (router.py가 파일 확장자(.py/.js)를 보고 각각 맞는 검사관 팀에 자동 배정. 둘 다 Semgrep 공통 보안 검사는 공유)
7. **Q**: Semgrep이 모든 언어에 공통으로 적용되는 이유는? **A**: (보안 취약점 패턴(OWASP Top 10)은 언어와 상관없이 존재. Python이든 JS든 SQL 인젝션은 위험하므로 항상 검사)
8. **Q**: run_tools 노드(Task 4-a)는 Task 3 도구들과 뭐가 달라? **A**: (Task 3 = 검사관 고용(함수 만들기). Task 4-a = 지휘자(LangGraph 노드로 묶어서 흐름에 연결, 언어 감지+병렬 실행+결과 수집))
9. **Q**: PATCH /issues/{id}/dismiss가 왜 /insights/issues/...가 아니라 /issues/...인가? **A**: (insights 라우터는 prefix가 /insights. dismiss는 /issues prefix 별도 라우터(api/issues.py)에 있음. 같은 파일에 넣으면 URL이 /insights/issues/...로 꼬임)

변경 이력

날짜	변경 내용	이유
2026-05-22 (초기)	전체 문서 작성 — Python 전용 5개 도구	기초 설계
2026-05-22 (1차)	Task 3 전면 재작성 — 다국어 지원 (Python/JS/TS/C#/HTML/CSS) + router.py 자동 배정	PRD 다국어 요구사항 반영

날짜	변경 내용	이유
2026-05-22 (2차)	Task 4-a 신규 추가 — run_tools 노드 (검사관 지휘자)	구현 계획에 있던 노드 누락 보완
2026-05-22 (2차)	Task 5 State 바구니 — `static_raw/security_raw` 설명을 다국어 도구로 업데이트	다국어 도구 변경에 따른 동기화
2026-05-22 (2차)	Task 6 — dismiss 전용 파일(`api/issues.py`) 분리 이유 설명 추가	URL prefix 충돌 문제 설명
2026-05-22 (2차)	요약표 — Task 4-a 행 추가, Task 5/6 설명 업데이트	구현 계획과 동기화
2026-05-22 (2차)	퀴즈 Q8/Q9 추가 — run_tools 노드, dismiss 라우터 분리	새 개념 이해 확인